



Module 6: Anomaly Detection

Agenda for Today

- Overview of Techniques for Anomaly Detection
- Statistical Techniques
- Supervised Techniques
- Unsupervised Techniques

A Problem: Data Validation

A Problem: Data Validation

- Charles was working on a problem of ingesting data into a system
- We knew what good data looked like, and we wanted to detect bad data to prevent corrupted data from being ingested
- The number of columns was large, so we couldn't write rules for each column

**Discuss: How you would tackle
this problem?**

Anomaly Detection is Hard



Anomalous != Bad

"Outliers are not necessarily a bad thing. These are just observations that are not following the same pattern as the other ones. But it can be the case that an outlier is very interesting. For example, if in a biological experiment, a rat is not dead whereas all others are, then it would be very interesting to understand why. This could lead to new scientific discoveries. So, it is important to detect outliers."

– Pierre Lafaye de Micheaux, Author and Statistician

Anomaly = Outlier

What is an outlier?

An **outlier** or **anomaly** is a data point which differs from the rest of the observations in a dataset.

**Anomaly Detection $\neq$ Novelty
Detection**

Anomaly Detection $\neq$ Novelty Detection

- Anomaly detection involves learning from data that has both inliers and outliers.
- Novelty detection involves learning starting with a dataset that does not have outliers

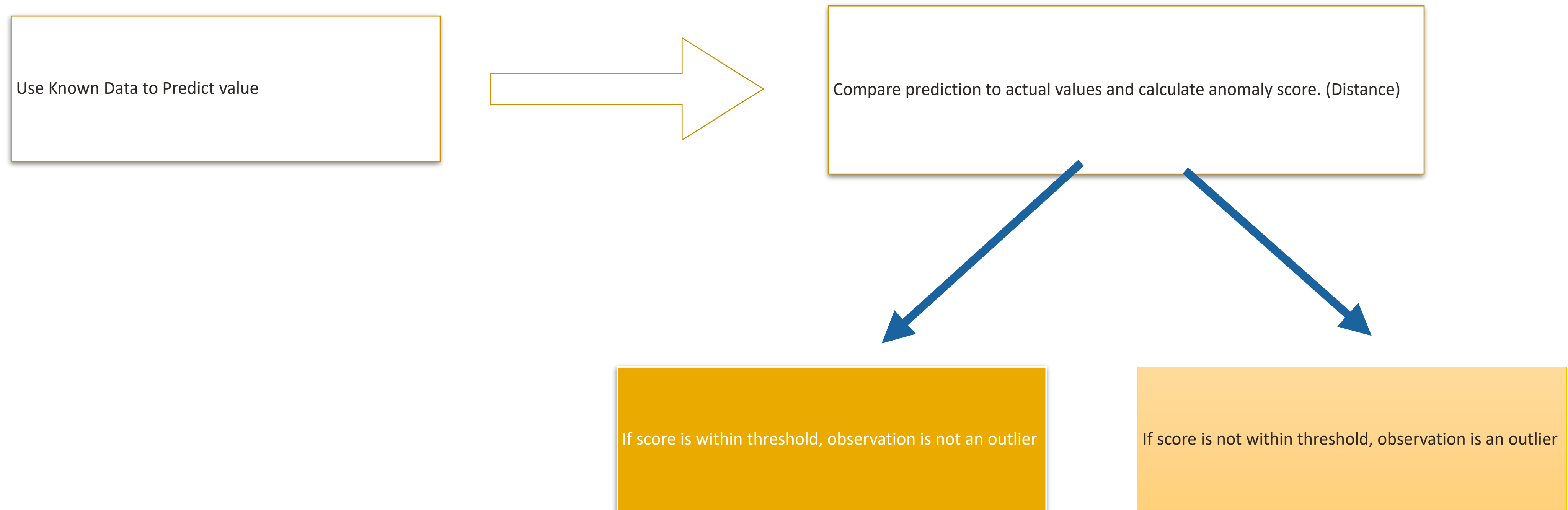
Categories of Anomaly Detection

- Forecasting (Supervised Learning)
- Statistical Metrics
- Unsupervised Techniques
- Density Based Methods
- Goodness of fit tests

Python Ecosystem

- **Pandas/SciPy/NumPy:** Pandas, SciPy and NumPy can be used for some of the simpler statistical calculations
- **PyOD:** A relatively new module for outlier detection that contains many sophisticated techniques. (<https://pyod.readthedocs.io/en/latest/>)
- **PMDArima:** A forecasting module intended to automate many of the tasks involved with ARIMA. (<http://alkaline-ml.com/pmdarima/about.html#about>)
- **Prophet:** A forecasting module that provides fast and automated forecasts which can be tuned by hand. (<https://facebook.github.io/prophet/>)
- **StatsModels:** A Python module that provides classes and functions for the estimation of many different statistical models, as well as for conducting statistical tests, and statistical data exploration. (<https://www.statsmodels.org/stable/index.html>)
- **Scikit-Learn:** Scikit-learn actually has several algorithms include the OneClassSVM and IsolationForest
- Others...

Anomaly Detection Process



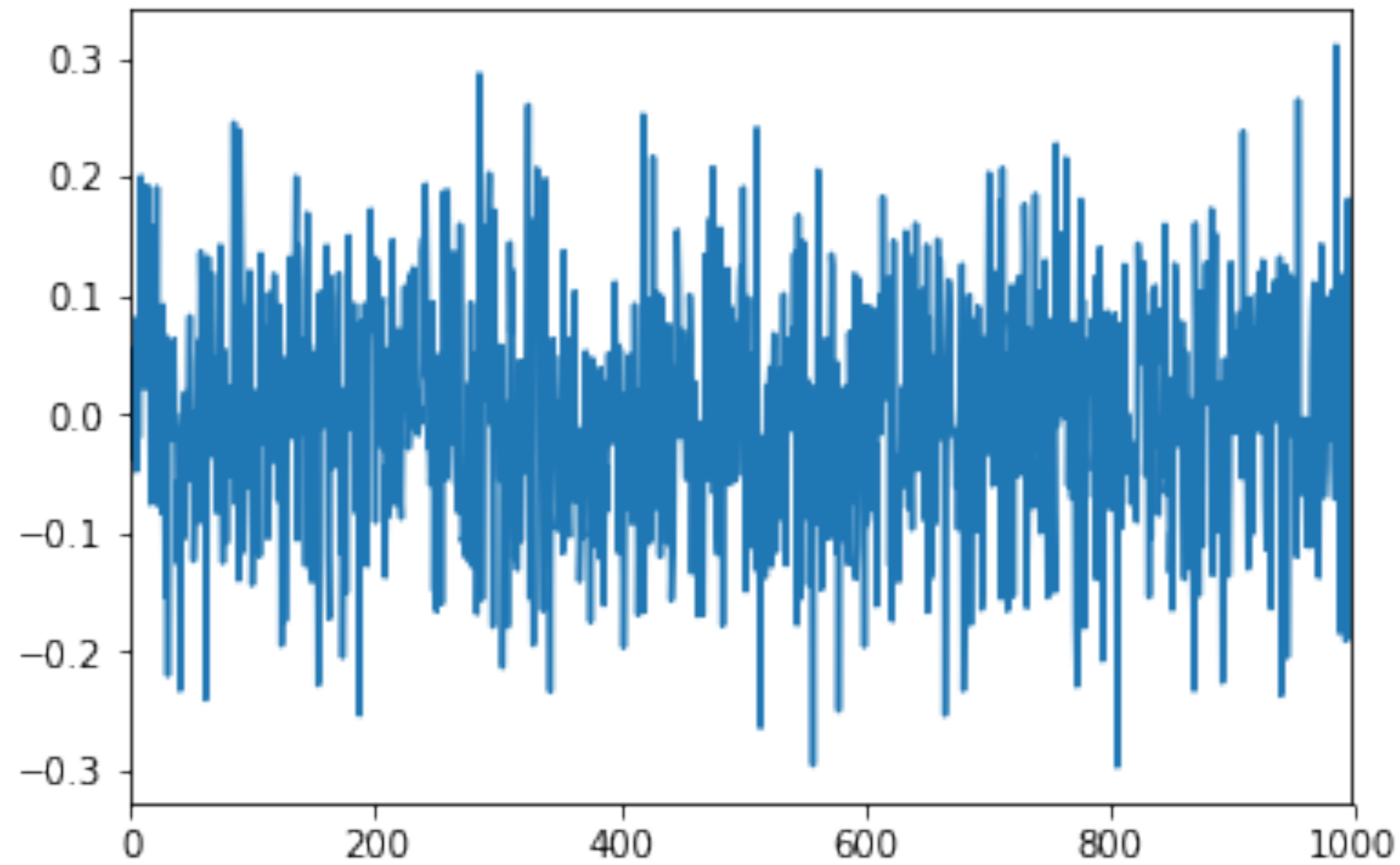
**Before you start... there are two
questions:**

**Is the dataset univariate or
multivariate?**

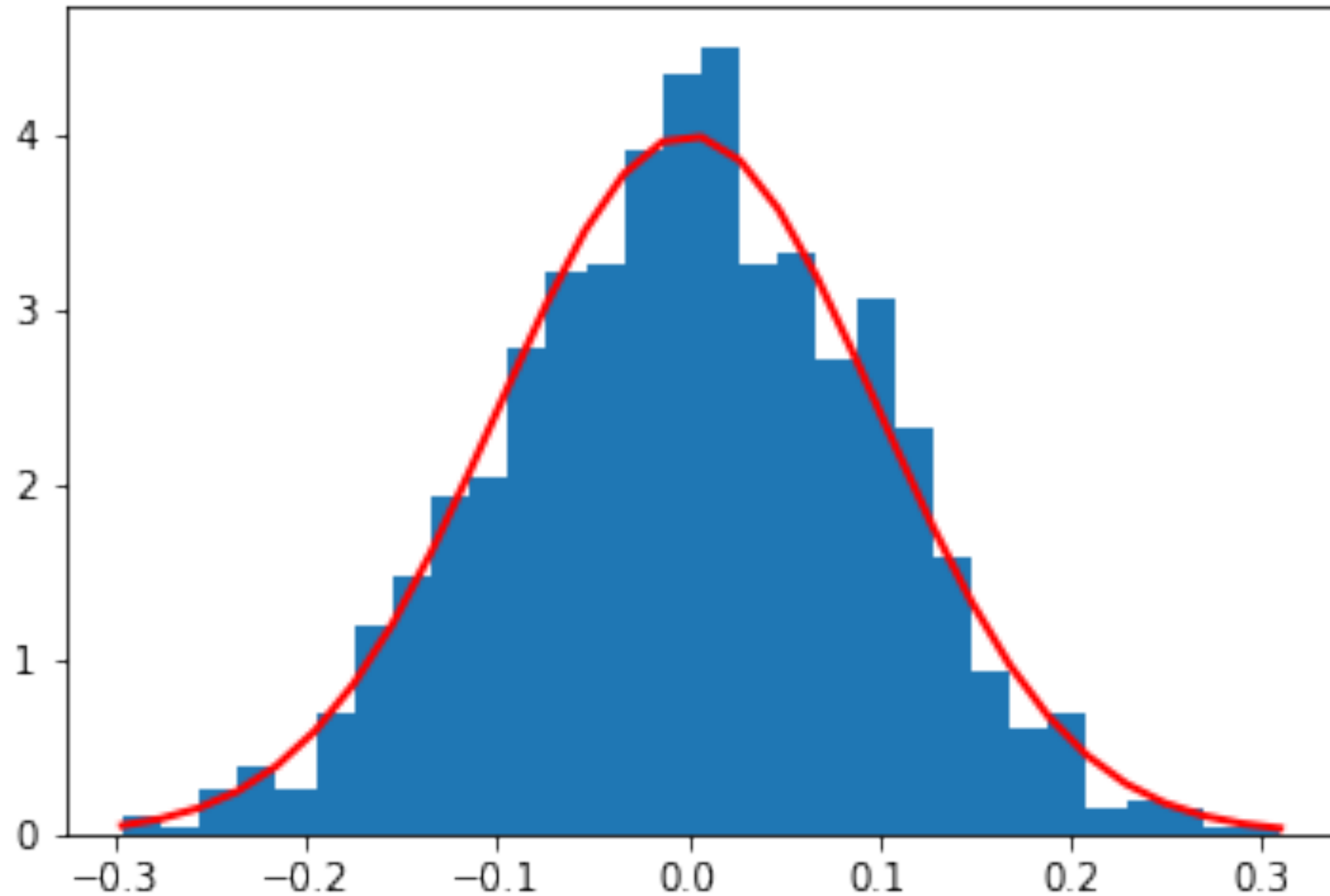
**What is the expected distribution
of my data, if any?**

Simple Statistics

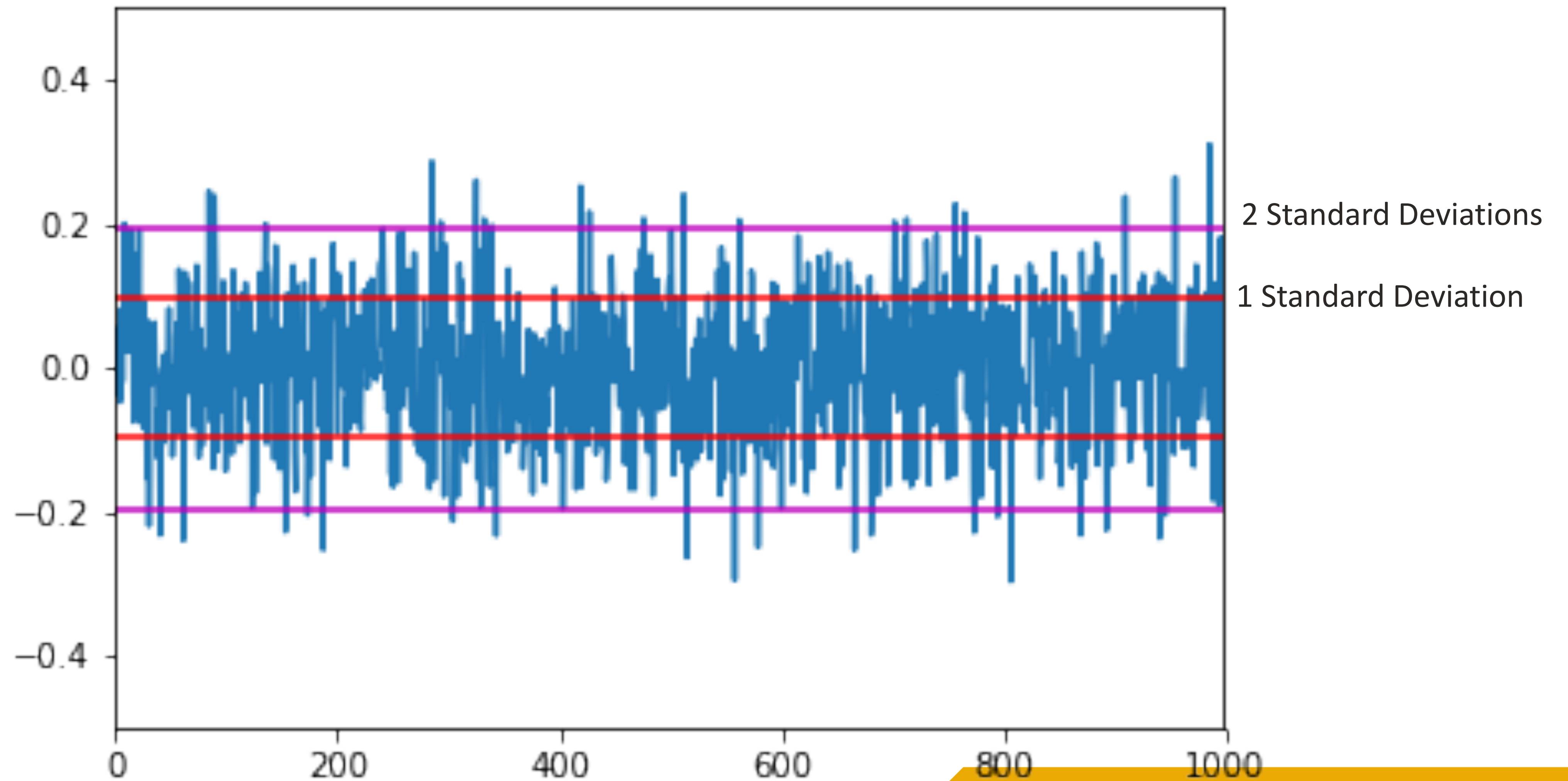
Simple Statistics



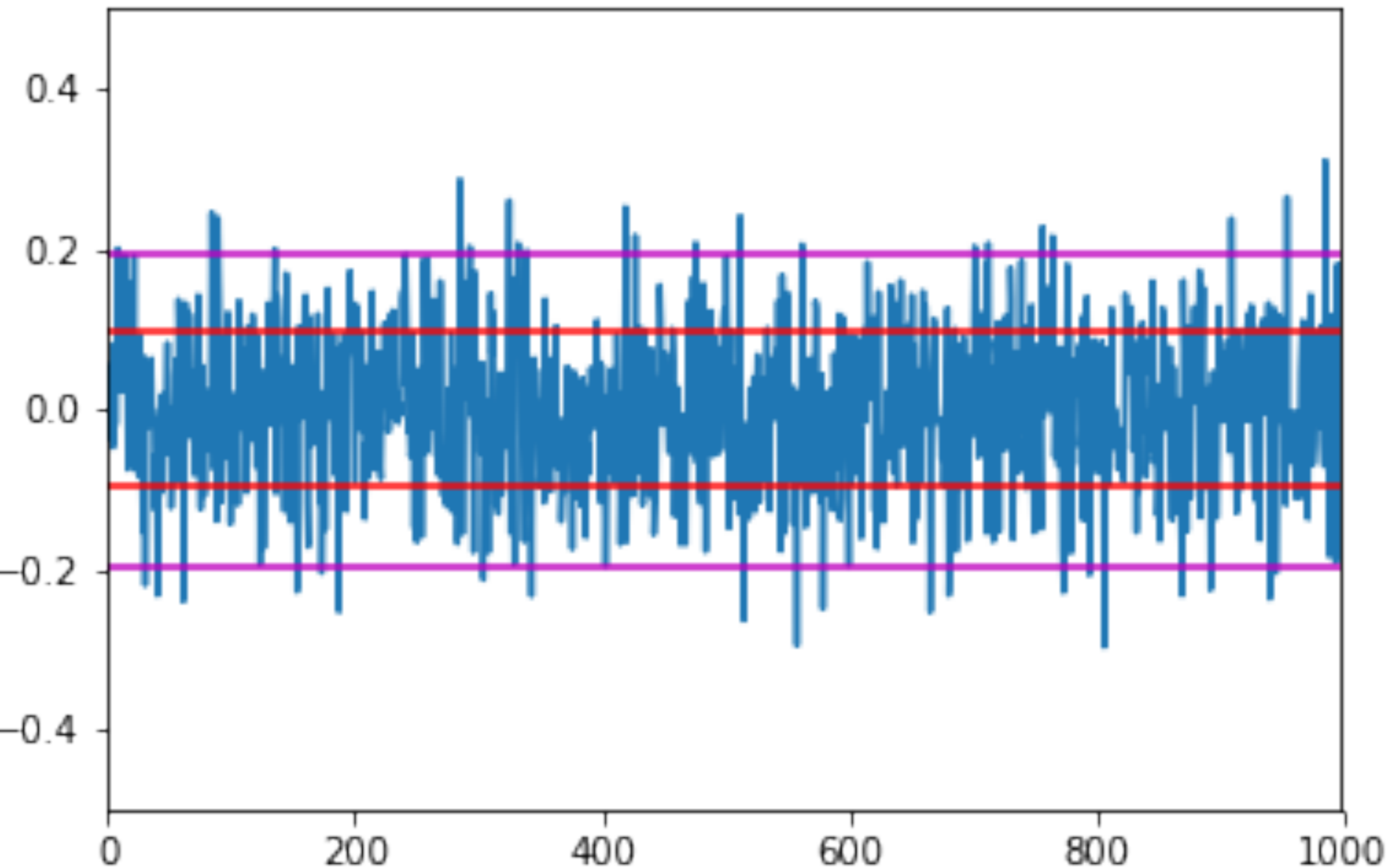
Simple Statistics



Simple Statistics

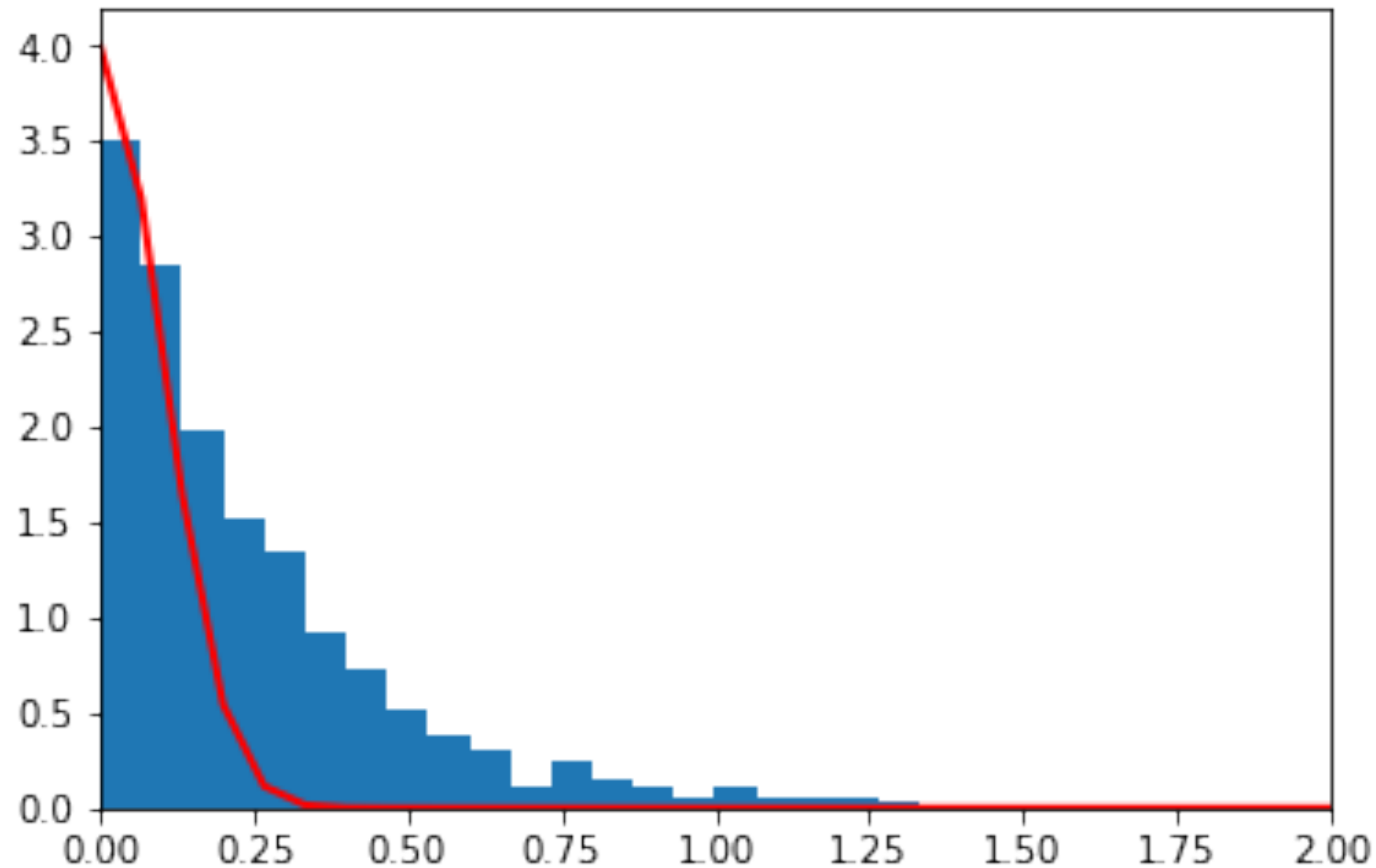


Simple Statistics



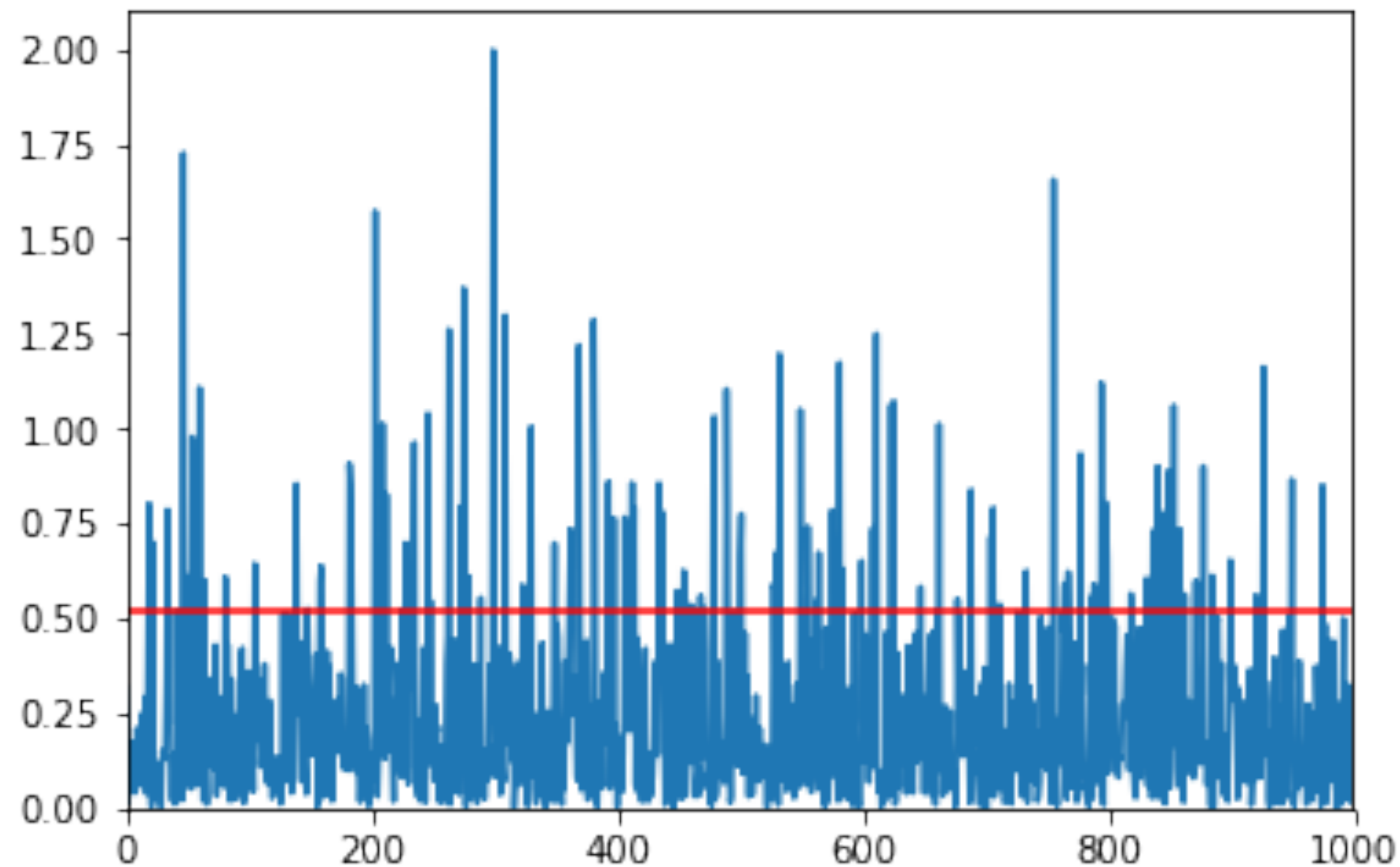
167 are outside of 1 deviation
44 are outside of 2 deviations

Simple Statistics



What if your data isn't distributed normally?

Simple Statistics



What if your data isn't distributed normally?

123 Outliers

Mean Absolute Deviation

$$\frac{\sum |x - \bar{x}|}{n}$$

`mad = sum(data_point - column_mean) / n`

- Mean Absolution Deviation (MAD) is defined as the average of the absolute deviations from a central point.

```
data = pd.Series(<data>)  
mad = data.mad()
```


Z-Score measures how many standard deviations an observation is from the mean.

Z-Score measures how many standard deviations an observation is from the mean.

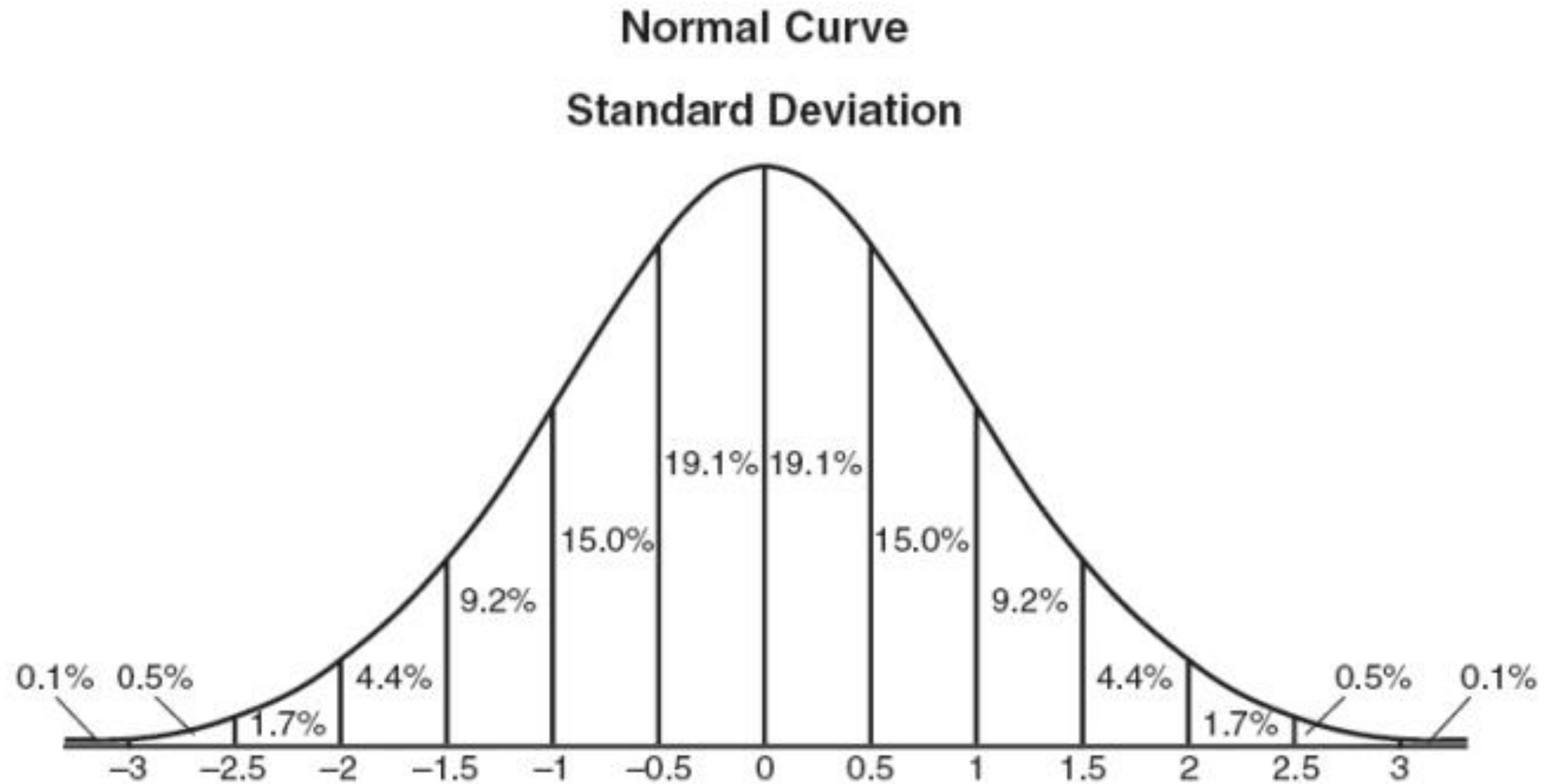
$$z = \frac{x - \mu}{\sigma}$$

```
z_score = (data_point - column_mean) / standard_deviation
```

```
#Use SciPy  
from scipy.stats import zscore  
df.apply(zscore)
```

```
#Use Pandas (Only works for all numeric cols)  
df_zscore = (df - df.mean())/df.std()
```


Z-Score



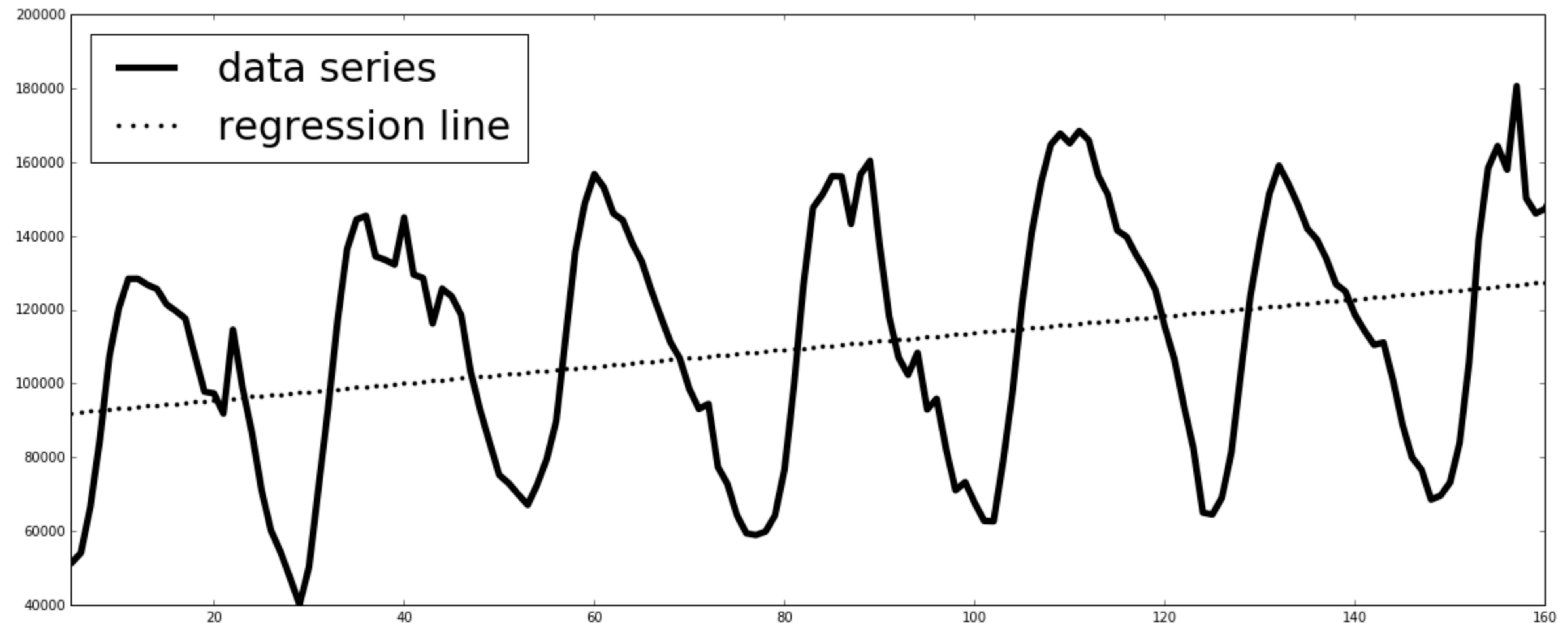
Z-Score pros:

- It is a very effective method if you can describe the values in the feature space with a gaussian distribution. (Parametric)
- The implementation is very easy using pandas and scipy.stats libraries.

Z-Score cons:

- It is only convenient to use in a low dimensional feature space, in a small to medium sized dataset.
- Is not recommended when distributions can not be assumed to be parametric.

What if your data looks like this?



Forecasting

Forecasting



Forecasting is a supervised learning technique which uses past observations to predict future events.

Forecasting is suited for single dimensional, time series data.

Terms

- **Trends:** Long term direction of changes in the data
- **Seasons:** Periodic repetitions of patterns in data
- **Cycles:** General changes in the data that have pattern similarities but vary in periodicity

ARIMA

Auto

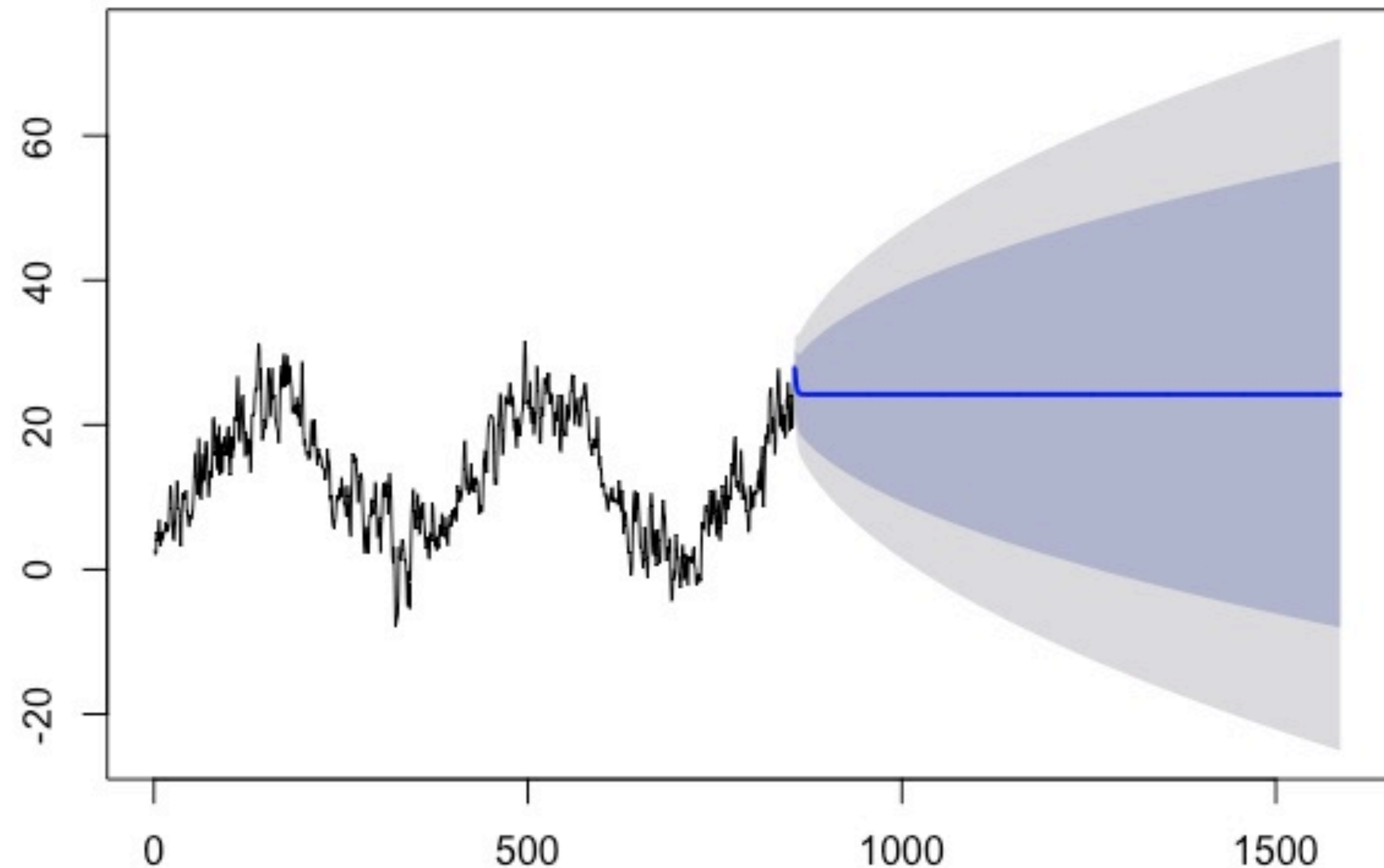
Regressive

Integrated

Moving

Average

Forecasts from ARIMA(1,1,2)



ARIMA

- **AR:** *Autoregression*. A model that uses the dependent relationship between an observation and some number of lagged observations.
- **I:** *Integrated*. The use of differencing of raw observations (e.g. subtracting an observation from an observation at the previous time step) in order to make the time series stationary.
- **MA:** *Moving Average*. A model that uses the dependency between an observation and a residual error from a moving average model applied to lagged observations.

ARIMA

The parameters of the ARIMA model are defined as follows:

- **p**: The number of lag observations included in the model, also called the lag order. (ar)
- **d**: The number of times that the raw observations are differenced, also called the degree of differencing. (integ)
- **q**: The size of the moving average window, also called the order of moving average. (ma)

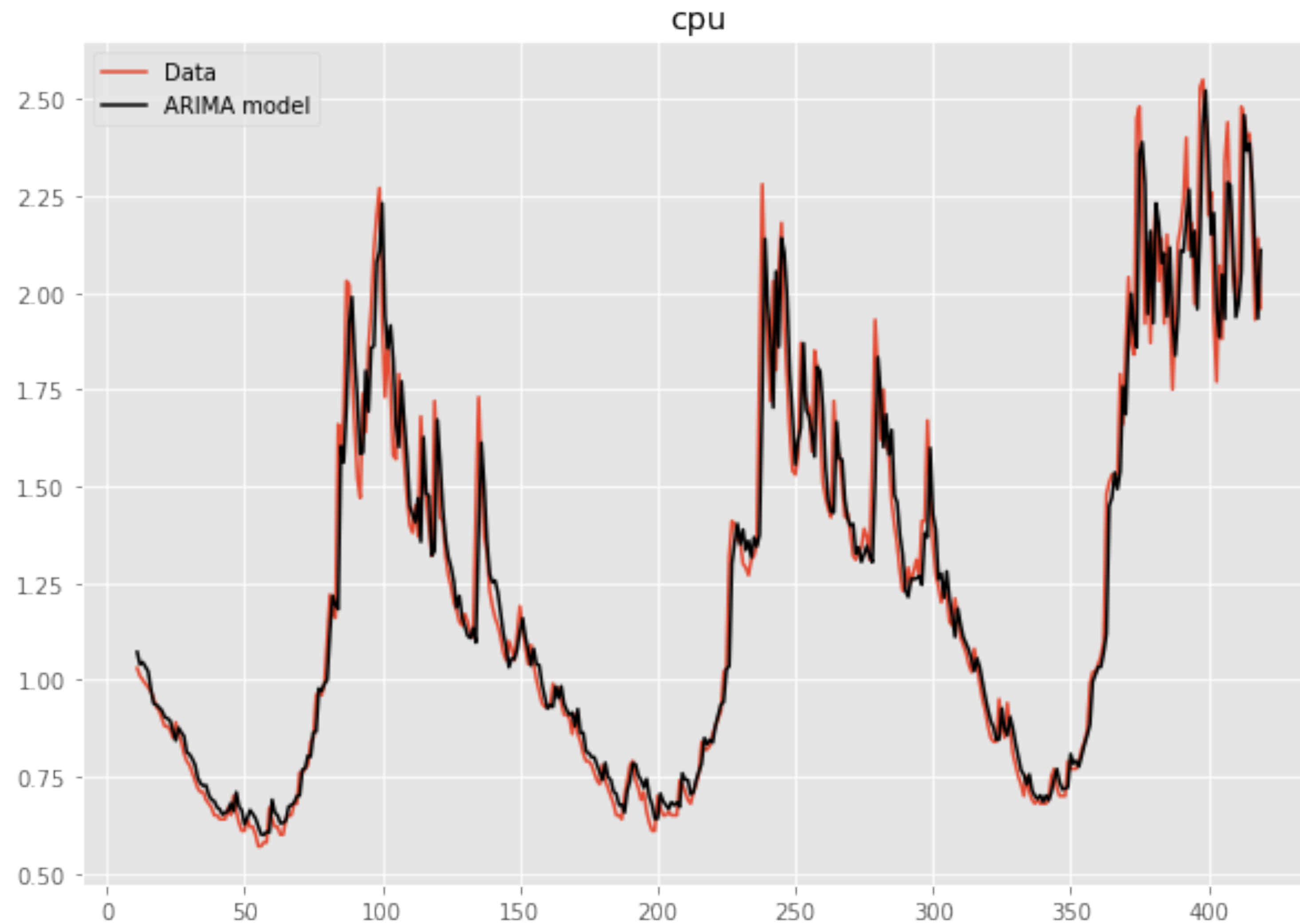
ARIMA

```
import pandas as pd
from pmdarima.arima import auto_arima
from pmdarima.arima import ADFTest

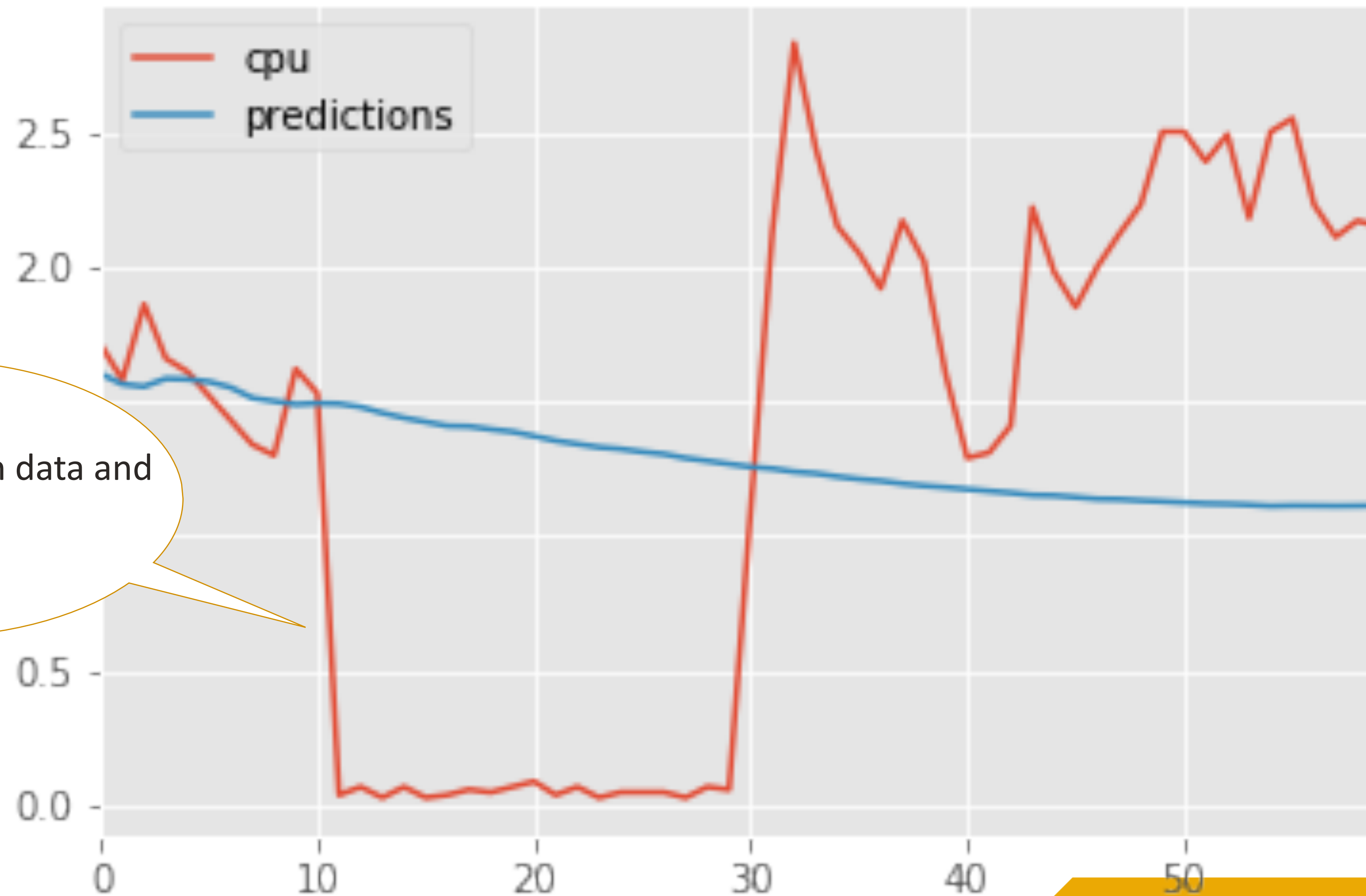
...

# Get data & build model
arima_model = auto_arima(df['cpu'], seasonal=True, error_action='ignore')
arima_model.summary()
```

ARIMA



ARIMA



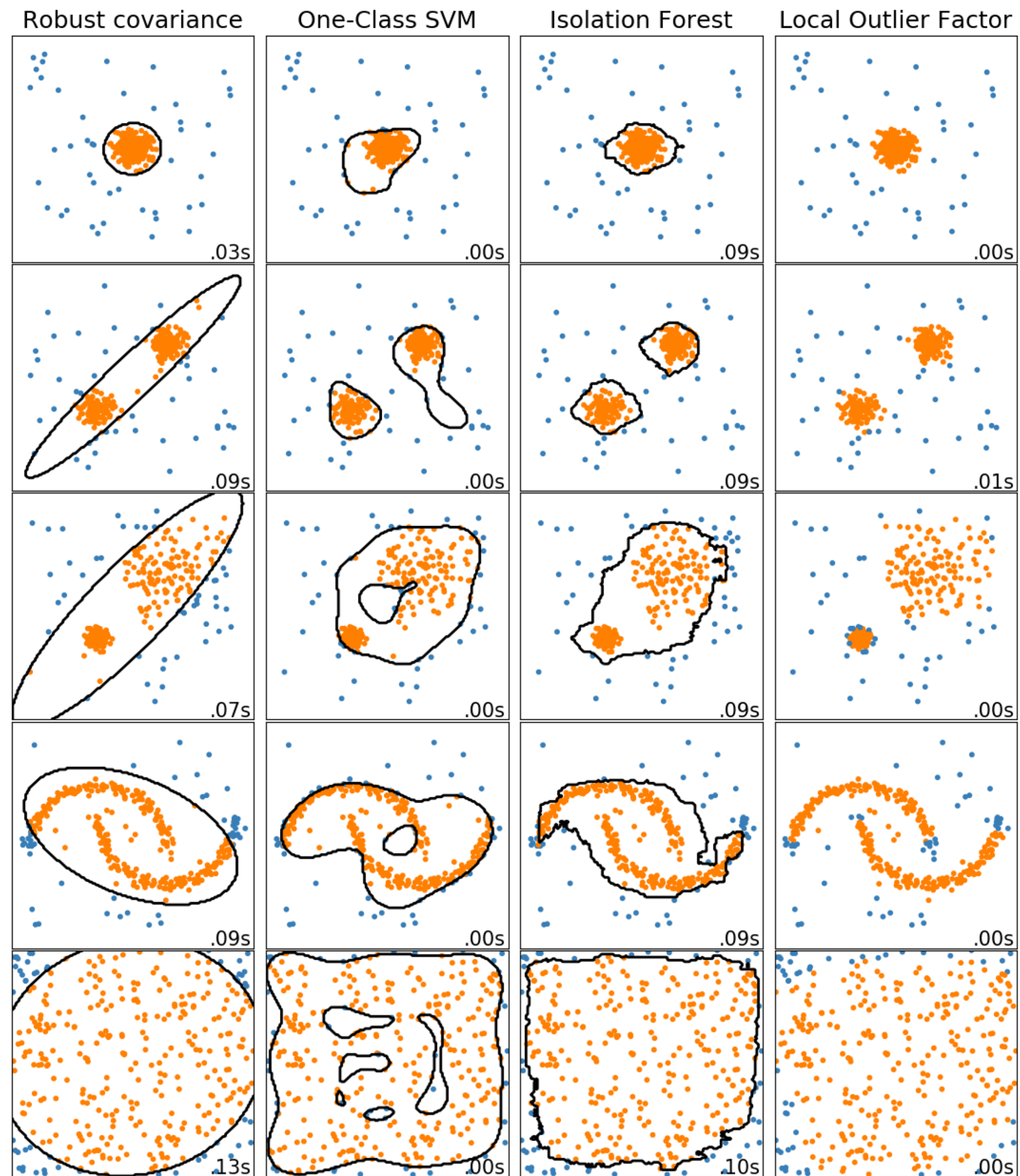
Large delta between data and prediction

ARIMA

- SARIMA: Seasonal ARIMA can be used for data with seasonally changing data
- ARIMAX: Another variation of ARIMA, commonly used for economic forecasting.

Questions?

Unsupervised Learning Techniques for Anomaly Detection



One Class SVM Implementation

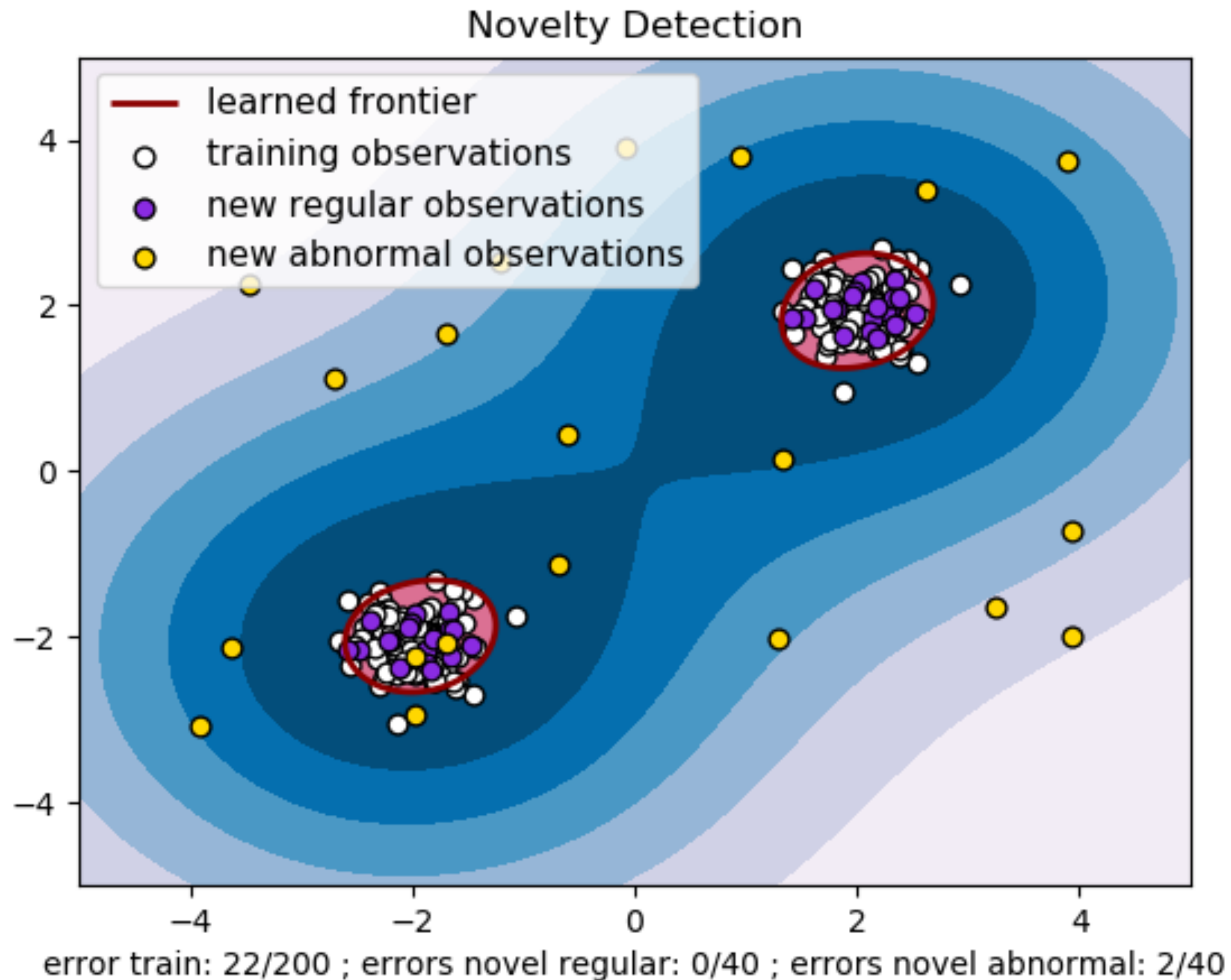
- The one class SVM can be used for novelty detection by creating one big class.
- If the observation is a member of the class, it is not an outlier

One Class SVM Implementation

```
from sklearn import svm

# fit the model
clf = svm.OneClassSVM(nu=0.1, kernel="rbf", gamma=0.1)
clf.fit(X_train)
y_pred_train = clf.predict(X_train)
```

One Class SVM Implementation



https://scikit-learn.org/stable/auto_examples/svm/plot_oneclass.html

One Class SVM Pros

- Useful for high dimensional data sets
- Does not require any assumptions about the distribution of the data

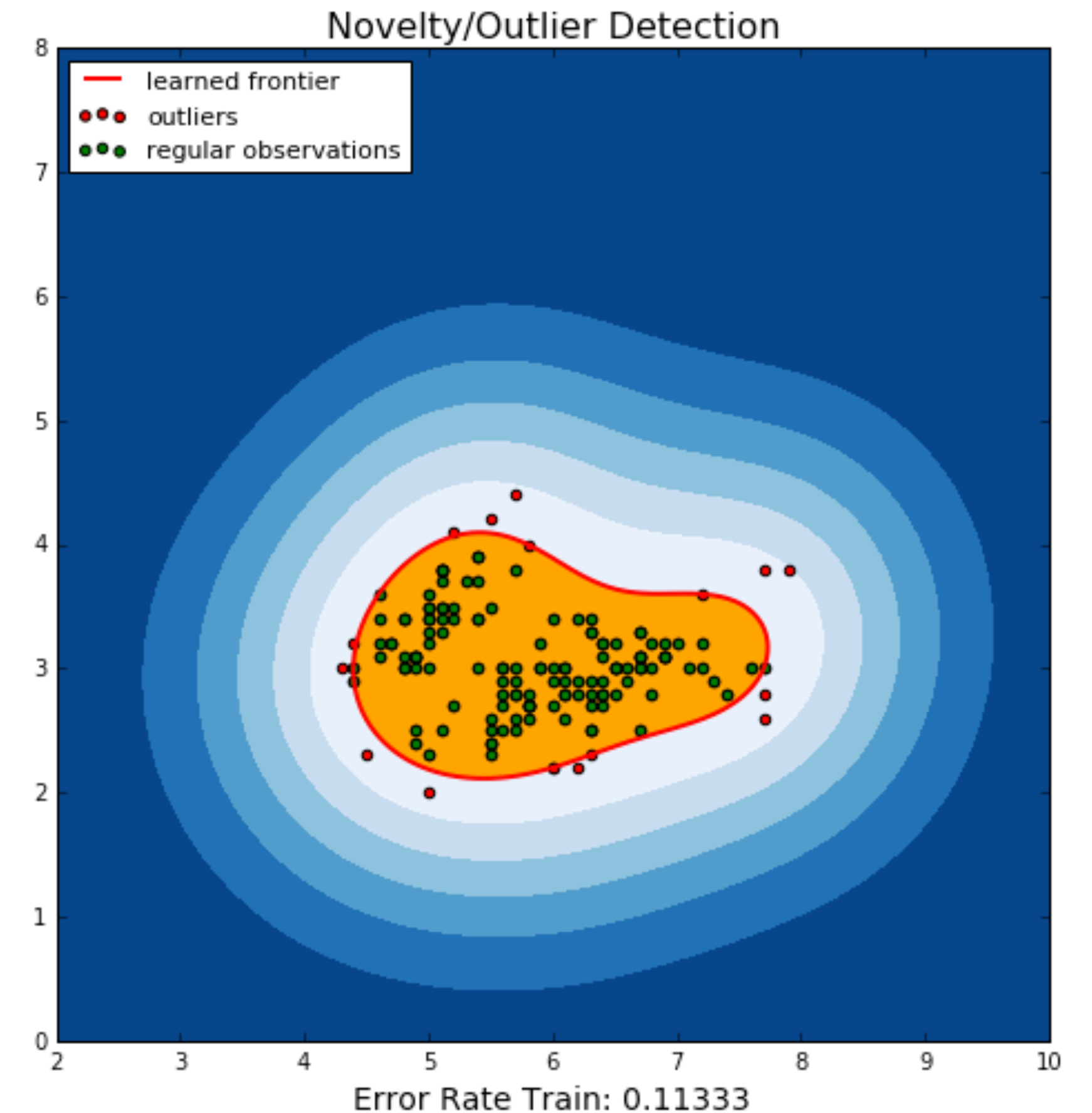
One Class SVM Cons

- Sensitive to outliers
- Best used for novelty detection rather than anomaly detection

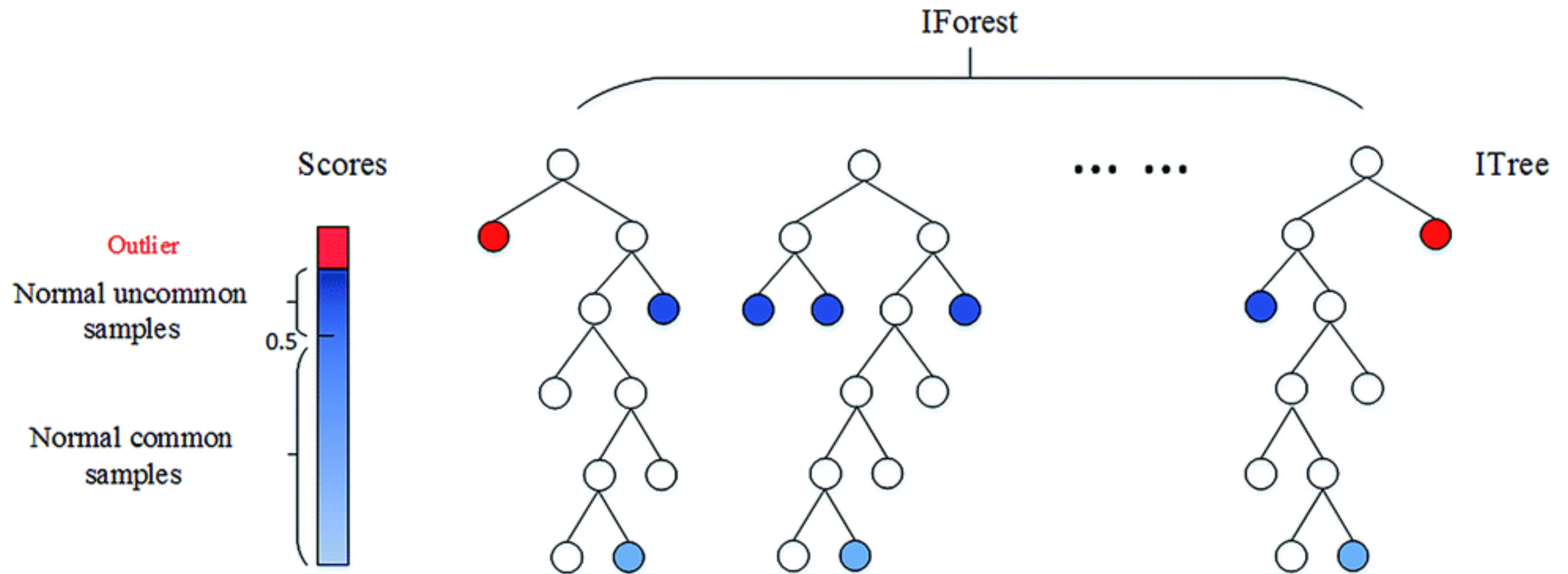
Outlier Detection

```
clf = svm.OneClassSVM( tol=0.001, nu=0.1)
clf.fit(X)
target_pred_outliers=clf.predict(X)
```

Delete n% of “outlier data”,
here ~10%



Isolation forest's basic principle is that outliers are few and far from the rest of the observations.



To Build an Isolation Forest

- Randomly pick a feature and a random split value between the max and min.
- Repeat for all observations in the training set
- A prediction is made by traversing the tree and measuring the path length. Shorter paths are likely to be outliers.

Isolation Forest Advantages

- There is no need of scaling the values in the feature space.
- It is an effective method when value distributions can not be assumed.
- It has few parameters, this makes this method fairly robust and easy to optimize.
- Scikit-Learn's implementation is easy to use and the documentation is superb.

Isolation Forest Disadvantages

- Visualizing results is complicated.
- If not correctly optimized, training time can be very long and computationally expensive.

Isolation Forest Implementation

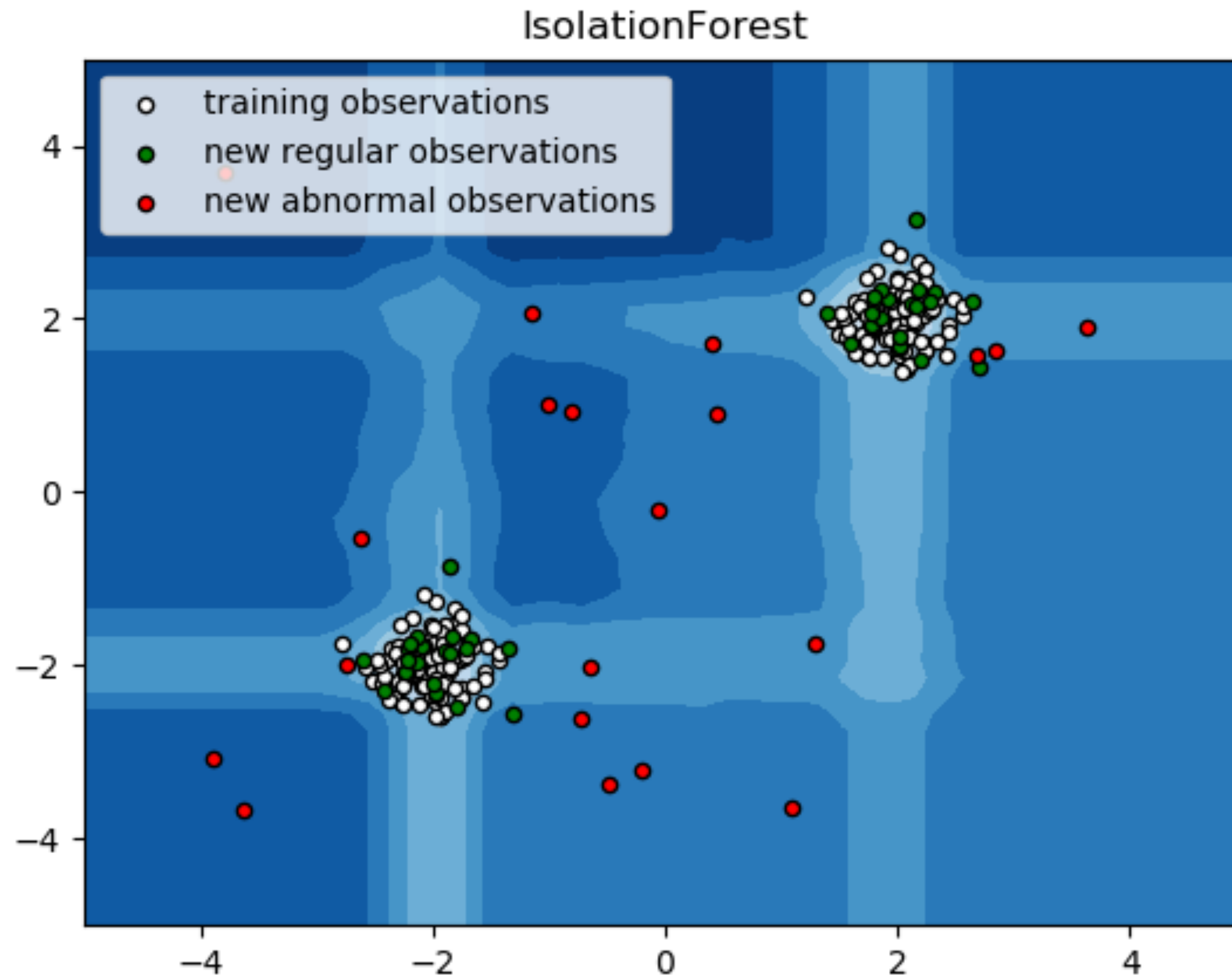
- Scikit Learn Documentation: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>

```
from sklearn.ensemble import IsolationForest

# fit the model
clf = IsolationForest(behavior='new', max_samples=100,
                      random_state=rng, contamination='auto')

clf.fit(X_train)
y_pred_train = clf.predict(X_train)
y_pred_test = clf.predict(X_test)
y_pred_outliers = clf.predict(X_outliers)
```

Isolation Forest Implementation



**Back to our data validation
problem...**

What if you could measure how similar a string is to another?

You can...

String Distance Functions

- **Cosine Similarity:** Converts strings to vectors and compares the angles of the two vectors.
- **Jaccard Distance:** Measures the dissimilarity between two sets.
- **Jaro Distance:** A similarity algorithm indicating the percentage of matched characters between two character sequences. The Jaro measure is the weighted sum of percentage of matched characters from each file and transposed characters.
- **Levenshtein Distance:** The number of changes needed to change one sequence into another, where each change is a single character modification (deletion, insertion, substitution)

String Distance Functions

- Let's say you have a dataset containing mac addresses in the format `XX:XX:XX:XX:XX`.
- What if you tokenized these a bit, replacing alpha numerics with A and leaving any symbols?

String Distance Functions

```
import pandas as pd
import re
import Levenshtein

good_macs = """DB:66:A7:58:0F:2F
38:A2:73:56:42:71
0B:C9:25:F7:E5:C8
1E:DF:B7:F4:D7:61
75:5E:35:61:FA:61
45:FD:F6:68:D0:2B
E2:1D:80:04:BB:09
56:A6:05:6B:F1:08
B3:B8:08:FD:36:21
24:06:E6:AD:8D:76
DF:8B:55:C7:71:9C""".split("\n")

unknown_macs = """FC:CF:FF:F8:1A:90
34:0E:03:F0:EE:19
7B:76:5D:5B:8E:FA
76:EE:88:D2:E6:B8
A7:17:2C:EA:79:E1
BB-1E-FB-A3-CE-DF
41:12:FD:84:07:0D
A3:43:2A:48:68:3A""".split("\n")

def tokenize_data(d):
    return re.sub('\w', 'A', mac)

for mac in unknown_macs:
    t = tokenize_data(mac)
    distance = Levenshtein.distance(t, template_token)
    if distance > 0:
        print("Anomaly found: ", mac, distance)
```

Anomaly found: BB-1E-FB-A3-CE-DF

Questions?

Group Discussion

Goal: Domain spoofing in links is a common tactic to fool unsuspecting users into clicking on links. For instance: faceboook.com or papajohns.com. (That is a Russian Rho, not a P).

In your groups, discuss how you might build a model to detect links like this. Let us assume you are given a list of URLs with just domains.

1. What data would you need for this?
2. Which modeling technique(s) would you try?
3. How would you measure success?

In Class Exercise

Complete Worksheet 6.1: Anomaly Detection.